INSTITUTO DE INFORMÁTICA
Universidade Federal de Goiás

Introdução à Programação

Aula 15 – Passagem por referência e Recursão

Prof. Jacson Rodrigues Barbosa

jacson_rodrigues@ufg.br

INSTITUTO DE INFORMÁTICA UFG

UFG UNIVERSIDADE FEDERAL DE GOIÁS

Definindo e utilizando Ponteiros em Go

- Exemplo ilustrativo de uso de ponteiros em movimentação bancária.
- Elementos de sintaxe quando usa um ponteiro para uma variável (operadores básicos):
  - “E comercial” (&): ao colocar um “E comercial” na frente de um nome de variável, você está declarando que quer obter o endereço ou um ponteiro para aquela variável.
  - Asterisco (*): operador de *desreferenciação* (valor da variável armazenado naquele endereço)

Código básico utilizando ponteiros

Passagem de Parâmetro por Valor e por Referência

- Existem duas formas de passarmos os valores dos argumentos para os parâmetros das funções:
  - Passagem por valor: O valor das variáveis dos argumentos não se alteram, mesmo que os parâmetros sejam modificado na função.
  - Passagem por referência: Os valores das variáveis dos parâmetros refletem nos argumentos.
- Passagem por referência, utilizamos o operador de referência (&)

Passagem de Parâmetro por Valor e por Referência

```
package main
import "fmt"
func main() {
    v := 0
    incrementa(v)
    fmt.Println(v)
}
func incrementa(x int) {
     x++
}
```

```
package main
import "fmt"
func main() {
      v := 0
      incrementa(&v)
      fmt.Println(v)
}
func incrementa(x *int) {
      *x++
}
```

Passagem de Parâmetro por Valor e por Referência

```
package main
import "fmt"
func main() {
    v := 0
    vf := 0
    vf = incrementa(v)
    fmt.Println(v) // 0
    fmt.Println(vf) // 1
}
func incrementa(x int) int {
    return x + 1
}
```

```
package main
import "fmt"
func main() {
    v := 0
    incrementa(&v)
    fmt.Println(v) // 1
}
func incrementa(x *int) {
    *x++
}
```

Recursão

- Recursão é um conceito em que uma função chama a si mesma por meios diretos ou indiretos.
- Cada chamada para a função recursiva é uma versão menor, de modo que converge em algum ponto.
- Cada função recursiva tem um caso base ou condição base que é a instrução executável final em recursão e interrompe chamadas adicionais.

Recursão na natureza

Recursão na natureza

Recursão no problema fatorial

- Assim, utilizando o processo recursivo para resolver o problema fatorial:

$$
\begin{aligned}
4! &= 4 * 3! \\
&= 4 * (3 * 2!) \\
&= 4 * (3 * (2 * 1!)) \\
&= 4 * (3 * (2 * (1 * 0!))) \\
&= 4 * (3 * (2 * (1 * 1))) \\
&= 4 * (3 * (2 * 1)) \\
&= 4 * (3 * 2) \\
&= 4 * 6 \\
&= 24
\end{aligned}
$$

Função fatorial recursiva

```
package main

import "fmt"


func fact(n int) int {
    if n == 0 {
        return 1
    }
    return n * fact(n-1)
}

func main() {
    fmt.Println(fact(7))
}
```

Resolva os exercícios abaixo utilizando recursão

1 - Dados dois números inteiros x e n, calcula o valor de x^n.

2- Calcule a soma de todos os valores de um array de reais.

3- Dado um array de inteiros e o seu número de elementos, inverta a posição dos seus elementos.

4- Um problema típico em ciência da computação consiste em converter um número da sua forma decimal para a forma binária.

Saiba mais

- Links
  - Tutorialspoint
  - Geeks for Geeks
  - Go by Example
  - A tour of GO

Referências

[1] Algoritmos, Busca e Ordenação, Rafael C. S. Schouery. 2023